

# EASNFW Requirements Specification v3

Henrique Sander Lourenço, João Colombari

August 17, 2026

## Contents

|          |                                      |          |
|----------|--------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                  | <b>4</b> |
| 1.1      | Purpose . . . . .                    | 4        |
| 1.2      | Scope . . . . .                      | 4        |
| 1.3      | Product Overview . . . . .           | 4        |
| 1.3.1    | Product Perspective . . . . .        | 4        |
| 1.3.2    | Product Functions . . . . .          | 5        |
| 1.3.3    | System Architecture . . . . .        | 5        |
| 1.3.4    | User Characteristics . . . . .       | 5        |
| 1.3.5    | Limitations . . . . .                | 6        |
| 1.4      | Definitions . . . . .                | 6        |
| 1.4.1    | Audio track . . . . .                | 6        |
| 1.4.2    | Audio processing algorithm . . . . . | 6        |
| 1.4.3    | Processed audio track . . . . .      | 6        |
| 1.4.4    | Environmental sample set . . . . .   | 6        |
| 1.4.5    | Self-test sequence . . . . .         | 6        |
| 1.4.6    | Payload . . . . .                    | 6        |
| <b>2</b> | <b>Firmware Images</b>               | <b>6</b> |
| 2.1      | Production image . . . . .           | 6        |
| 2.2      | Development image . . . . .          | 7        |
| 2.3      | Debug image . . . . .                | 7        |
| 2.4      | HIL verification image . . . . .     | 7        |
| <b>3</b> | <b>Data Types</b>                    | <b>7</b> |
| 3.1      | Ecoacoustic Record . . . . .         | 7        |
| 3.2      | Power-on Log . . . . .               | 7        |
| <b>4</b> | <b>Requirements</b>                  | <b>7</b> |
| 4.1      | Initialization . . . . .             | 7        |
| 4.1.1    | REQ-001 . . . . .                    | 7        |
| 4.2      | Data Acquisition . . . . .           | 8        |
| 4.2.1    | REQ-002 . . . . .                    | 8        |
| 4.2.2    | REQ-003 . . . . .                    | 8        |
| 4.2.3    | REQ-004 . . . . .                    | 8        |
| 4.3      | Data Processing . . . . .            | 8        |
| 4.3.1    | REQ-005 . . . . .                    | 8        |
| 4.4      | Cloud Communication . . . . .        | 8        |
| 4.4.1    | REQ-006 . . . . .                    | 8        |
| 4.4.2    | REQ-007 . . . . .                    | 8        |
| 4.4.3    | REQ-008 . . . . .                    | 8        |

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| 4.5      | Data Persistence . . . . .                  | 9         |
| 4.5.1    | REQ-009 . . . . .                           | 9         |
| 4.5.2    | REQ-010 . . . . .                           | 9         |
| 4.6      | Power Management . . . . .                  | 9         |
| 4.6.1    | REQ-011 . . . . .                           | 9         |
| 4.7      | Self-test Sequence . . . . .                | 9         |
| 4.7.1    | REQ-012 . . . . .                           | 9         |
| 4.7.2    | REQ-013 . . . . .                           | 9         |
| 4.7.3    | REQ-014 . . . . .                           | 9         |
| 4.7.4    | REQ-015 . . . . .                           | 9         |
| 4.7.5    | REQ-016 . . . . .                           | 9         |
| 4.7.6    | REQ-017 . . . . .                           | 9         |
| 4.7.7    | REQ-018 . . . . .                           | 9         |
| 4.7.8    | REQ-019 . . . . .                           | 9         |
| <b>5</b> | <b>Design Constraints and Observability</b> | <b>10</b> |
| 5.1      | Communication Parameters . . . . .          | 10        |
| 5.2      | Configuration . . . . .                     | 10        |
| 5.3      | Observability . . . . .                     | 10        |
| <b>6</b> | <b>Verification</b>                         | <b>10</b> |
| 6.1      | Verification Plan . . . . .                 | 10        |
| 6.1.1    | TC-001 (REQ-001) . . . . .                  | 11        |
| 6.1.2    | TC-002 (REQ-002) . . . . .                  | 11        |
| 6.1.3    | TC-003 (REQ-003) . . . . .                  | 11        |
| 6.1.4    | TC-004 (REQ-004) . . . . .                  | 11        |
| 6.1.5    | TC-005 (REQ-005) . . . . .                  | 11        |
| 6.1.6    | TC-006 (REQ-006) . . . . .                  | 11        |
| 6.1.7    | TC-007 (REQ-007) . . . . .                  | 11        |
| 6.1.8    | TC-008 (REQ-008) . . . . .                  | 11        |
| 6.1.9    | TC-009 (REQ-009) . . . . .                  | 12        |
| 6.1.10   | TC-010 (REQ-010) . . . . .                  | 12        |
| 6.1.11   | TC-011 (REQ-011) . . . . .                  | 12        |
| 6.1.12   | TC-012 (REQ-002) . . . . .                  | 12        |
| 6.1.13   | TC-013 (REQ-002, REQ-005) . . . . .         | 12        |
| 6.1.14   | TC-014 (REQ-012) . . . . .                  | 12        |
| 6.1.15   | TC-015 (REQ-013) . . . . .                  | 12        |
| 6.1.16   | TC-016 (REQ-014) . . . . .                  | 12        |
| 6.1.17   | TC-017 (REQ-015) . . . . .                  | 12        |
| 6.1.18   | TC-018 (REQ-016) . . . . .                  | 12        |
| 6.1.19   | TC-019 (REQ-017) . . . . .                  | 13        |
| 6.1.20   | TC-020 (REQ-018) . . . . .                  | 13        |
| 6.1.21   | TC-021 (REQ-019) . . . . .                  | 13        |
| 6.1.22   | TC-022 (REQ-001, REQ-007) . . . . .         | 13        |
| 6.1.23   | TC-023 (REQ-001, REQ-009) . . . . .         | 13        |
| 6.1.24   | TC-024 (REQ-002, REQ-004) . . . . .         | 13        |
| 6.1.25   | TC-025 (REQ-002, REQ-003) . . . . .         | 13        |
| 6.1.26   | TC-026 (REQ-006, REQ-008) . . . . .         | 13        |
| 6.1.27   | TC-027 (REQ-012 to REQ-019) . . . . .       | 14        |
| 6.1.28   | TC-028 (REQ-011) . . . . .                  | 14        |
| 6.1.29   | TC-029 (REQ-010) . . . . .                  | 14        |
| 6.1.30   | TC-030 (REQ-009) . . . . .                  | 14        |

|        |        |           |    |
|--------|--------|-----------|----|
| 6.1.31 | TC-031 | . . . . . | 14 |
| 6.1.32 | TC-033 | . . . . . | 14 |

# 1 Introduction

## 1.1 Purpose

The purpose of the software whose requirements are specified in this document is to control the hardware of an ecoacoustic sensing node.

## 1.2 Scope

This document specifies the requirements for the software product designated as EASNFW, from **ecoacoustic sensing node firmware**.

Within this document, EASNFW may also be referred to as *the firmware*. The ecoacoustic sensing node as a whole, comprising all software and hardware components of the device, is designated as EASN and may also be referred to as *the system* or *the sensing node*.

The purpose of EASNFW is to control the hardware of EASN in order to:

- acquire ecoacoustic record through the system sensors;
- process the acquired data; and
- transmit the acquired and processed data to a cloud platform.

The cloud platform may be referred to as *the cloud platform* or simply *the cloud*. The acquired data is intended for use in ecoacoustic studies.

## 1.3 Product Overview

### 1.3.1 Product Perspective

The system is intended to integrate a distributed ecoacoustic record acquisition infrastructure. This infrastructure is composed of a set of sensing nodes and a cloud platform.

The cloud platform stores the data acquired by the sensing nodes and provides support for visualizing and analyzing the collected data.

The system hardware is based on the boards nRF5340 Audio DK and nRF9151 SMA DK from Nordic Semiconductor.

EASNFW will run on and control the nRF5340 System-on-Chip (SoC) from the nRF5340 Audio DK and the nRF9151 System-in-Package (SiP) from the nRF9151 SMA DK. EASNFW is hence subdivided into two different components: one that will run on nRF5340 and be mainly responsible for interfacing with the system sensors, and processing, storing and retrieving the data acquired through them, and is therefore designated as EASNFW-SENSOR; and another one that will run on nRF9151 and be mainly responsible for interfacing with the cloud, and is therefore designated as EASNFW-CLOUD. EASNFW-SENSOR and EASNFW-CLOUD will communicate over SPI, with EASNFW-SENSOR as the controller and EASNFW-CLOUD as the peripheral.

EASNFW-CLOUD will communicate with the cloud using HTTP over LTE-M through the built-in LTE-M modem of nRF9151. Communication between EASNFW-CLOUD and the cloud platform is assymetric, in a way that only EASNFW-CLOUD can initiate the communication.

**User interfaces** EASNFW-SENSOR will provide a command line interface (CLI) for verifying specific features. Such interface will only be enabled when working with the “CLI verification image” (see Section 2).

Both EASNFW-SENSOR and EASNFW-CLOUD will log their own activities through the USB interface of their respective boards when logging is enabled.

### 1.3.2 Product Functions

The main functions that will be performed by EASNFW are described in Table 1, alongside the firmware component responsible for each function.

| Function                                                                   | Responsible firmware component |
|----------------------------------------------------------------------------|--------------------------------|
| Acquire audio data                                                         | EASNFW-SENSOR                  |
| Process audio data                                                         | EASNFW-SENSOR                  |
| Acquire environmental data                                                 | EASNFW-SENSOR                  |
| Transmit acquired environmental data and processed audio data to the cloud | EASNFW-CLOUD                   |

Table 1: EASNFW functions description

### 1.3.3 System Architecture

The diagram in Figure 1 provides an overview of the system architecture, showing the main blocks that compose and communicate with it, and how they interface with EASNFW.

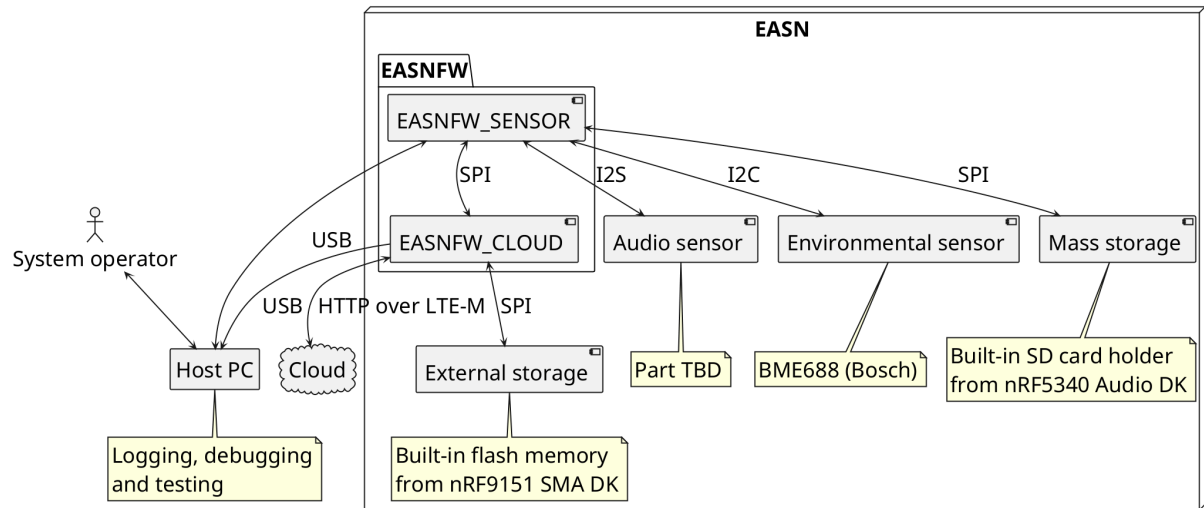


Figure 1: System architecture overview

### 1.3.4 User Characteristics

Users of EASN and EASNFW are expected to be familiar with general concepts related to:

- embedded systems;
- Internet of Things (IoT);
- wireless communication; and
- digital signal processing (DSP).

### 1.3.5 Limitations

EASNFW functions and their implementations are constrained by:

- the computational, memory, communication, and energy resources of the target hardware; and
- the interfaces provided by the cloud platform.

## 1.4 Definitions

### 1.4.1 Audio track

An ordered collection of audio samples captured sequentially over a continuous period of `PARAM_TRACK_LEN` seconds, which will be configurable via `Kconfig`.

### 1.4.2 Audio processing algorithm

The algorithm used to process the recorded audio tracks.

### 1.4.3 Processed audio track

The output, or collection of outputs, produced by the audio processing algorithm for a given audio track.

### 1.4.4 Environmental sample set

A set of environmental variables sampled within a short interval. The set consists of temperature, humidity, pressure and volatile organic compounds (VOCs).

### 1.4.5 Self-test sequence

Sequence of tests executed by EASNFW to verify the correct working of individual software modules, as well as the integration between them. It is broken down into the independent self-test sequence, which is a firmware component-specific sequence, meaning that each of EASNFW-SENSOR and EASNFW-CLOUD has its own independent self-test sequence; and the shared self-test sequence, which is carried out by EASNFW-SENSOR and EASNFW-CLOUD together and tests functionalities that depend on the interaction between the two components.

### 1.4.6 Payload

A payload is a collection of data transmitted by EASNFW to the cloud in a single message.

## 2 Firmware Images

Build options will be provided to allow different firmware images to be built depending on the intended usage.

### 2.1 Production image

Image of the firmware to be used for deployment. It executes all the business logic automatically out of reset, and has logging and other debug functionalities disabled.

## 2.2 Development image

Image of the firmware to be used for development. It is the same as the production image but has logging and other debug functionalities enabled.

## 2.3 Debug image

Image of the firmware to be used for debugging. It is the same as the development image, but with increased logging verbosity.

## 2.4 HIL verification image

Image of the firmware to be used for hardware-in-the-loop (HIL) verification. Instead of automatically executing the business logic out of reset, it runs a CLI that receives and parses commands through the USB interface of nRF5340 Audio DK, triggers specific code paths and returns results. Its purpose is to support the execution of tests and on-demand data acquisition and processing for verification and experimentation.

# 3 Data Types

This section defines different data types that EASNFW will handle.

## 3.1 Ecoacoustic Record

An ecoacoustic record comprises a processed audio track, a single sample of each tracked environmental variable, and a timestamp that indicates the date and time when the sampling of the audio track began. Ecoacoustic records are transmitted to the cloud through ecoacoustic record payloads. An ecoacoustic record might be organized into multiple ecoacoustic record payloads to allow for more flexibility in handling the transmission to the cloud.

## 3.2 Power-on Log

A power-on log consists of the respective revisions of EASNFW-SENSOR and EASNFW-CLOUD, the reason why the device was last powered off and the results of the self-test sequence. Power-on logs are transmitted to the cloud through power-on log payloads.

# 4 Requirements

## 4.1 Initialization

### 4.1.1 REQ-001

When the system is powered on, the firmware shall execute the initialization sequence, which comprises: (a) initializing critical peripherals<sup>1</sup>; (b) running the self-test sequences; (c) establishing an Internet connection; (d) transmitting the power-on log payload to the cloud; and (e) transmitting to the cloud any pending ecoacoustic record payloads from previous boot cycles.

---

<sup>1</sup>(a) is already taken care of by Zephyr.

## 4.2 Data Acquisition

### 4.2.1 REQ-002

After the initialization sequence is successfully completed, the firmware shall periodically sample the audio into PARAM\_TRACK\_LEN-second audio tracks with a PARAM\_INTER\_TRACK\_INTER second interval between the sampling of two consecutive audio tracks.

Parameters for audio sampling:

- Number of channels: 1
- Sample rate: PARAM\_AUDIO\_SAMPLE\_RATE kHz
- Bandwidth: PARAM\_AUDIO\_SAMPLE\_BW\_LO Hz - PARAM\_AUDIO\_SAMPLE\_BW\_HI kHz
- Sample width: PARAM\_AUDIO\_SAMPLE\_WIDTH bits

### 4.2.2 REQ-003

The firmware shall detect the loss of any audio samples and restart the audio track sampling in such an event.

### 4.2.3 REQ-004

Before the beginning of the sampling of each audio track, the firmware shall capture a timestamp, including the date and time with at least a 1-second resolution, and sample the environmental data.

## 4.3 Data Processing

### 4.3.1 REQ-005

The firmware shall process the audio tracks according to the audio processing algorithm.

## 4.4 Cloud Communication

### 4.4.1 REQ-006

After an audio track is processed, the firmware shall send the processed audio track, alongside the timestamp and environmental data that were captured before the beginning of the sampling of that audio track, to the cloud.

### 4.4.2 REQ-007

When the transmission of the power-on log payload to the cloud fails, the firmware shall retry transmitting it up to PARAM\_NUM\_RETRY\_TX\_CLOUD times with an exponential backoff model defined by  $t[s] = 2^c$ , where  $c$  is the number of times that the transmission has been retried.

### 4.4.3 REQ-008

When the transmission of an ecoacoustic record payload to the cloud fails, the firmware shall retry transmitting it until the free space of the mass storage is insufficient to store the data relative to another audio track. The firmware shall use an exponential backoff model defined by  $t[s] = 2^c$ , where  $c$  is the number of times that the transmission has been retried, with an upper limit of PARAM\_MAX\_INTERVAL\_BETWEEN\_RETRIES seconds between retries.



## **4.5 Data Persistence**

### **4.5.1 REQ-009**

The firmware shall persist ecoacoustic record payloads that have not been sent to the cloud across boot cycles.

### **4.5.2 REQ-010**

The firmware shall persist details of fatal failures across boot cycles.

## **4.6 Power Management**

### **4.6.1 REQ-011**

When the firmware is waiting for triggers to take action, the firmware shall enter an energy-saving state.

## **4.7 Self-test Sequence**

### **4.7.1 REQ-012**

The self-test sequence shall verify that audio data can be sampled from the audio sensor.

### **4.7.2 REQ-013**

The self-test sequence shall verify that environmental data can be sampled from the environmental sensor.

### **4.7.3 REQ-014**

The self-test sequence shall verify that data can be written to and read from the mass storage.

### **4.7.4 REQ-015**

The self-test sequence shall verify that data can be written to and read from the external storage.

### **4.7.5 REQ-016**

The self-test sequence shall verify the communication between EASNFW-SENSOR and EASNFW-CLOUD.

### **4.7.6 REQ-017**

The self-test sequence shall verify that data can be transmitted to the cloud.

### **4.7.7 REQ-018**

The self-test sequence shall verify that the reason for the last power-off can be retrieved.

### **4.7.8 REQ-019**

The self-test sequence shall log the results of the executed tests over USB for debugging purposes.

## 5 Design Constraints and Observability

### 5.1 Communication Parameters

The firmware will be designed so that communication parameters can be adapted to the regulatory requirements of different geographical regions.

### 5.2 Configuration

The following parameters will be configurable at compile-time via Kconfig:

- `PARAM_TRACK_LEN` (seconds)
- `PARAM_INTER_TRACK_INTERVAL` (seconds)
- `PARAM_NUM_RETRY_TX_CLOUD`
- `PARAM_AUDIO_SAMPLE_RATE` (kHz)
- `PARAM_AUDIO_SAMPLE_BW_LO` (Hz)
- `PARAM_AUDIO_SAMPLE_BW_HI` (kHz)
- `PARAM_AUDIO_SAMPLE_WIDTH` (bits)
- `PARAM_MAX_INTERVAL_BETWEEN_RETRIES` (seconds)

### 5.3 Observability

The firmware will log information relevant to verifying correct system operation and to identifying and diagnosing failures, in a way that the logs can be monitored in real-time from a host personal computer (PC) through USB.

## 6 Verification

Requirements might be verified through analysis of:

- firmware logs;
- data received by the cloud platform;
- data returned by the CLI verification image; and/or
- source code.

The firmware product will include a report with details about the verification performed for each requirement.

### 6.1 Verification Plan

The test cases described below shall be supported by the HIL verification image. Each test case is a code path implemented in the HIL verification image, exposed through the CLI, and is associated with the requirement it verifies.

#### 6.1.1 TC-001 (REQ-001)

Power on the system with the HIL verification image and issue the command to trigger the initialization sequence. Verify through firmware logs that the critical peripherals are initialized, the self-test sequences are run, an Internet connection is established, the power-on log payload is transmitted to the cloud, and any pending ecoacoustic record payloads from previous boot cycles are transmitted.

#### 6.1.2 TC-002 (REQ-002)

Issue the command to start periodic audio sampling. Verify through the returned data and firmware logs that audio tracks of PARAM\_TRACK\_LEN seconds are sampled with a PARAM\_INTER\_TRACK\_INTERVAL-second interval between consecutive tracks, and that the sampled tracks conform to the configured parameters (1 channel, PARAM\_AUDIO\_SAMPLE\_RATE kHz sample rate, PARAM\_AUDIO\_SAMPLE\_BW\_LO Hz to PARAM\_AUDIO\_SAMPLE\_BW\_HI kHz bandwidth, and PARAM\_AUDIO\_SAMPLE\_WIDTH bits sample width).

#### 6.1.3 TC-003 (REQ-003)

Issue the command to start audio sampling and induce a loss of audio samples. Verify through firmware logs that the sample loss is detected and that the audio track sampling is restarted.

#### 6.1.4 TC-004 (REQ-004)

Issue the command to sample an audio track. Verify through the returned data that a timestamp with at least 1-second resolution and a sample of each environmental variable (temperature, humidity, pressure and VOCs) were captured before the beginning of the audio track sampling.

#### 6.1.5 TC-005 (REQ-005)

Issue the command to sample and process an audio track. Verify that the returned processed audio track conforms to the audio processing algorithm.

#### 6.1.6 TC-006 (REQ-006)

Issue the command to sample, process and transmit an audio track. Verify through the received data by the cloud platform that the processed audio track, the timestamp and the environmental data are correctly received.

#### 6.1.7 TC-007 (REQ-007)

Simulate a failure in the transmission of the power-on log payload. Verify through firmware logs that the firmware retries the transmission up to PARAM\_NUM\_RETRY\_TX\_CLOUD times with an exponential backoff model defined by  $t[s] = 2^c$ .

#### 6.1.8 TC-008 (REQ-008)

Simulate a failure in the transmission of an ecoacoustic record payload. Verify through firmware logs that the firmware retries the transmission with an exponential backoff model defined by  $t[s] = 2^c$  until the free space of the mass storage is insufficient to store the data relative to another audio track.

#### **6.1.9 TC-009 (REQ-009)**

Issue the command to create an ecoacoustic record payload and simulate a transmission failure so the payload remains unsent. Power cycle the system and verify through firmware logs and the data received by the cloud platform that the pending payload is transmitted after the new boot.

#### **6.1.10 TC-010 (REQ-010)**

Trigger a fatal failure and power cycle the system. Verify through firmware logs that the details of the fatal failure are still available after the boot.

#### **6.1.11 TC-011 (REQ-011)**

Issue the command to enter the waiting state. Verify through firmware logs that the firmware enters an energy-saving state while waiting.

#### **6.1.12 TC-012 (REQ-002)**

Issue the CLI command to sample an audio track with a specified length, passed as a parameter. Verify that the captured samples are returned.

#### **6.1.13 TC-013 (REQ-002, REQ-005)**

Issue the CLI command to sample an audio track with a specified length, passed as a parameter, and process it according to the audio processing algorithm. Verify that both the captured samples and the processed audio track are returned.

#### **6.1.14 TC-014 (REQ-012)**

Issue the command to run the self-test sequence. Verify that the test for sampling audio data from the audio sensor passes.

#### **6.1.15 TC-015 (REQ-013)**

Issue the command to run the self-test sequence. Verify that the test for sampling environmental data from the environmental sensor passes.

#### **6.1.16 TC-016 (REQ-014)**

Issue the command to run the self-test sequence. Verify that the test for writing to and reading from the mass storage passes.

#### **6.1.17 TC-017 (REQ-015)**

Issue the command to run the self-test sequence. Verify that the test for writing to and reading from the external storage passes.

#### **6.1.18 TC-018 (REQ-016)**

Issue the command to run the self-test sequence. Verify that the test for the communication between EASNFW-SENSOR and EASNFW-CLOUD passes.

#### **6.1.19 TC-019 (REQ-017)**

Issue the command to run the self-test sequence. Verify that the test for transmitting data to the cloud passes.

#### **6.1.20 TC-020 (REQ-018)**

Issue the command to run the self-test sequence. Verify that the test for retrieving the reason for the last power-off passes.

#### **6.1.21 TC-021 (REQ-019)**

Issue the command to run the self-test sequence. Verify that the results of the executed tests are logged over USB.

#### **6.1.22 TC-022 (REQ-001, REQ-007)**

Power on the system with the HIL verification image and simulate a failure in the transmission of the power-on log payload during the initialization sequence. Verify through firmware logs that the firmware retries the transmission up to `PARAM_NUM_RETRY_TX_CLOUD` times with an exponential backoff model defined by  $t[s] = 2^c$ , and that the remaining steps of the initialization sequence are executed.

#### **6.1.23 TC-023 (REQ-001, REQ-009)**

Power on the system with the HIL verification image with pending ecoacoustic record payloads from a previous boot cycle. Issue the command to trigger the initialization sequence. Verify through firmware logs and the data received by the cloud platform that the pending payloads are transmitted to the cloud.

#### **6.1.24 TC-024 (REQ-002, REQ-004)**

Issue the command to start periodic audio sampling. Verify through the returned data that a timestamp with at least 1-second resolution and a sample of each environmental variable (temperature, humidity, pressure and VOCs) are captured before the beginning of the sampling of each audio track.

#### **6.1.25 TC-025 (REQ-002, REQ-003)**

Issue the command to start periodic audio sampling and induce a loss of audio samples. Verify through firmware logs that the sample loss is detected, that the audio track sampling is restarted, and that the periodic sampling continues with the subsequent tracks.

#### **6.1.26 TC-026 (REQ-006, REQ-008)**

Issue the command to sample, process and transmit an audio track and simulate a failure in the transmission of the resulting ecoacoustic record payload. Verify through firmware logs that the firmware retries the transmission of the data until the free space of the mass storage is insufficient to store the data relative to another audio track.

#### **6.1.27 TC-027 (REQ-012 to REQ-019)**

Issue the command to run the full self-test sequence. Verify that all tests pass: sampling audio data from the audio sensor, sampling environmental data from the environmental sensor, writing to and reading from the mass storage, writing to and reading from the external storage, the communication between EASNFW-SENSOR and EASNFW-CLOUD, transmitting data to the cloud, retrieving the reason for the last power-off, and that the results are logged over USB.

#### **6.1.28 TC-028 (REQ-011)**

Issue the command to enter the waiting state and then, while still in the waiting state, issue a trigger. Verify through firmware logs that the firmware enters an energy-saving state while waiting and exits it when the trigger arrives.

#### **6.1.29 TC-029 (REQ-010)**

Trigger a fatal failure and verify through firmware logs that the details of the fatal failure are logged.

#### **6.1.30 TC-030 (REQ-009)**

Issue the command to create multiple ecoacoustic record payloads, and simulate transmission failures so that the payloads remain unsent. Power cycle the system and verify through firmware logs and the data received by the cloud platform that the payloads are transmitted after the new boot.

#### **6.1.31 TC-031**

Issue the CLI command to retrieve the firmware version. Verify that the returned version matches the expected version.

#### **6.1.32 TC-033**

Issue the CLI command to check the free space of the mass storage. Verify that the returned value matches the expected free space.